GenSpark

What is Full Stack Development

If you've spent any time around software teams, you've probably heard someone described as a "full stack developer." It's one of the most commonly used — and loosely used — terms in the industry. Understanding what it actually means will help you understand where your own skills fit and what's expected of you as you grow.

The Two Sides of an Application

Almost every application you interact with, whether it's a banking app, an online store, or an internal company tool, is built from two broad layers.

The frontend is everything the user sees and interacts with directly — the buttons, forms, screens, and visual layout. When you fill out a login form or click "submit" on an order, you're interacting with the frontend. Frontend work typically involves languages and tools like HTML, CSS, and JavaScript, along with frontend frameworks (React, Angular, Vue, and others) that make building complex interfaces more manageable.

The backend is everything that happens behind the scenes to make that frontend actually work. When you click "submit" on that order form, something has to check your payment info, save the order to a database, and send a confirmation. That's backend work — servers, databases, business logic, and APIs (the mechanisms that let the frontend and backend talk to each other). Backend work is typically done in languages like Python, Java, C#, or Node.js, and involves databases like PostgreSQL, MySQL, or MongoDB.

So What's "Full Stack"?

A full stack developer is someone who can work across both layers — frontend and backend — rather than specializing in just one. A full stack developer can build the login form *and* the server logic that checks the password *and* the database that stores the user's account.

This doesn't mean a full stack developer is equally expert in every technology across both layers. In practice, most full stack developers have a stronger side (frontend-leaning or backend-leaning) and

enough working knowledge of the other side to be effective without needing to hand off every task to a specialist. The value isn't perfect mastery of everything — it's the ability to understand and contribute to the whole flow of a feature, from the screen a user sees to the data that gets stored.

Why This Matters for an FDE

The Forward Deployed Engineer role tends to demand full stack thinking more than many other engineering roles, for a practical reason: you're often working directly with a client, sometimes without a large team of specialists standing between you and the problem. If a client's data isn't showing up correctly, you may need to trace the issue yourself — is it a frontend display bug, a broken API call, or a backend/database problem? Being able to reason across the entire stack, rather than immediately escalating to someone else, is part of what makes this role effective.

You won't be expected to be a world-class expert in every layer immediately. But you will be expected to develop enough fluency across the stack that you can diagnose problems, build features end-to-end, and communicate clearly with specialists when you do need to bring one in.

A Simple Mental Model

Here's a way to think about a full stack application, using an online expense report tool as an example:

- **Frontend**: The web page where an employee fills out an expense form and clicks submit.
- **API layer**: The connection point where the frontend sends that form data somewhere to be processed.
- **Backend**: The server logic that validates the expense, checks approval rules, and decides what happens next.
- **Database**: Where the expense record actually gets stored so it can be retrieved later.

A full stack developer understands how data flows through all four of these pieces, even if their day-to-day work is concentrated in one or two of them.

Comprehension Check

1. In your own words, what's the difference between frontend and backend work?
2. Why might full stack skills be especially valuable in a client-facing engineering role like FDE?
3. Using the expense report example, what would you check first if an employee said their

submitted expense "disappeared" — and why?

← Previous: SDLC overview: phases, roles, and process

Next: Coding best practices →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!